

FleetFlow ERP — Project Documentation

A complete technical and functional overview: what the project is for, how it is built, and what every module does.

Audience. Project stakeholders, new engineers, and reviewers who need one authoritative description of the system. **Companion documents:** [README](#) (quick start & architecture), [PRD-STATUS](#) (contracted-scope coverage map), [VERIFICATION-GUIDE](#) (how to test each module), [DATABASE](#) (table definitions), [API](#) (endpoints), [DEPLOY](#) and [MIGRATION](#).

Table of contents

1. [Project objective](#)
2. [What the system delivers](#)
3. [Technology stack](#)
4. [Architecture & implementation approach](#)
5. [Cross-cutting foundations](#)
6. [Module reference — functions in detail](#)
7. [Security](#)
8. [Quality & testing approach](#)
9. [External integrations](#)
10. [Delivery status](#)
11. [Repository structure](#)

1. Project objective

FleetFlow is an integrated ERP for cross-border road-freight operators working the East & Central Africa trade corridors — Tanzania, Kenya, Uganda, Rwanda, Zambia and the DRC.

The problem it solves

Regional freight operators typically run their business across disconnected tools — spreadsheets for bookings, messaging apps for dispatch, and a separate accounting package that is reconciled late and by hand. The consequences are structural:

- **Operations and finance are siloed.** A single job is captured several times — once as a booking, again as trip costs, again as an invoice — with no shared record, so the numbers rarely agree.
- **Cross-border complexity is unmanaged.** Multiple currencies, five tax regimes, and a stack of compliance documents (insurance, inspection, COMESA, Yellow Card) are tracked informally.
- **There is no single source of truth.** Profitability *per trip*, *per corridor*, or *per customer* is guesswork, and month-end is painful.

The goal

Provide **one connected system** in which every operational action keeps the books correct automatically. Concretely, the project set out to:

1. **Unify the freight lifecycle** — booking → trip execution → invoicing → cash — on top of a genuine **double-entry accounting core**, so operations and finance can never drift apart.
2. **Run the whole business**, not just dispatch: fleet & driver compliance, inventory & procurement, statutory payroll, HR, fixed assets, a workshop, sales/CRM, retail POS, and analytics.

3. **Support multi-country, multi-company operations** from a single installation, with each legal entity's data strictly isolated.
4. **Make the data trustworthy** — balanced and immutable accounting, an audit trail, optimistic concurrency, and enforced access control.
5. **Be ready to connect** to the region's external services (mobile money, e-invoicing, bank feeds, GPS telematics) as credentials become available.

Scope

The contracted scope is **36 modules across 6 delivery phases. 33 are delivered**; the remaining three (Vendor Collaboration portal, Production Control, Questionnaire) are out of scope for a logistics operation. External integrations are modeled and ready but not yet switched on — see [§9](#) and [PRD-STATUS](#) for the exact map.

2. What the system delivers

Area	Capabilities
Freight operations	Cross-border orders, trip execution with per-trip P&L, dispatch with conflict-checking, master planning, delivery jobs
Fleet & compliance	Vehicle & driver registries, maintenance, document-expiry tracking, fuel-efficiency monitoring
Finance	Double-entry ledger, AP/AR, budgets, tax (VAT/WHT), multi-currency + revaluation, consolidation, collections, period close
Inventory & supply chain	Stock with moving-average costing, warehouses & transfers, procurement with 3-way match, product attributes, cost variance
Human capital	Statutory payroll, HR (contracts, leave, documents), time & attendance, expense claims
Assets & workshop	Fixed-asset register with depreciation, custody & warranty, service orders
Commercial	CRM leads, freight quotations that convert to orders, retail POS, project/contract management
Analytics	KPI dashboard across five categories, corridor P&L, dock/damage/feedback capture
System	Multi-company registry, users, dynamic roles & permissions, audit log, demo sandbox

At a glance: 78 data models · 96 permissions · 5 built-in roles (plus custom) · ~50 operational screens · 266 automated tests.

3. Technology stack

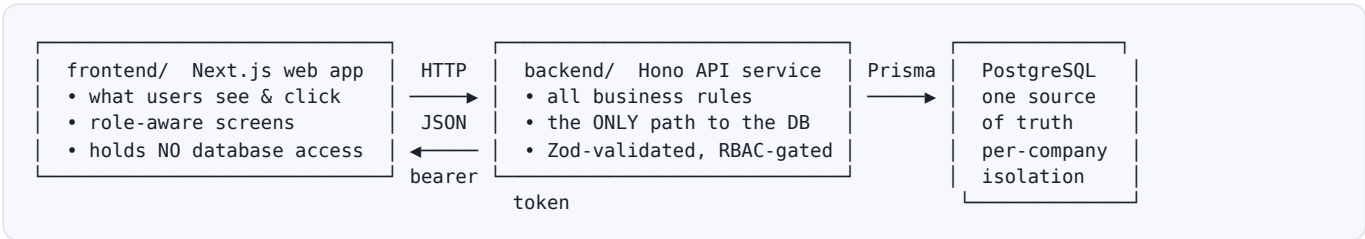
Layer	Technology	Notes
Web frontend (frontend/)	Next.js 15 (App Router), React 19, TypeScript	Tailwind CSS + shadcn/ui, React Hook Form, Zod. No database dependency.
API backend (backend/)	Hono (Node server), TypeScript	Framework-independent domain services; Zod validation; owns all DB access.
Database	PostgreSQL	Accessed only through Prisma ORM (parameterized queries; no raw SQL in app paths).
ORM	Prisma 6	78 models, 35 migrations.
Authentication	NextAuth (Auth.js v5) + JWT (jose)	Web session bridges a stateless bearer token issued by the API.
Accounting	In-house double-entry engine	Decimal -based, multi-currency, balance-enforced.
Documents	PDFKit (PDF), PapaParse (CSV)	Dispatch sheets, daily reports, payment PDFs, CSV exports — on the backend.
Email	nodemailer	Password-reset and notifications.
Testing	Vitest	35 unit-test files over the domain services, plus integration tests.
Passwords	bcrypt (cost 12)	

Both apps are **independently built and deployed** — there is no monorepo manifest; each has its own `package.json` , `lockfile` , `tsconfig` , and `.env` .

4. Architecture & implementation approach

4.1 Two independently-deployable services

The system is split into two apps that communicate over HTTP:



The defining rule: **the backend owns all data**. The web app never touches the database directly — it only calls the API with a bearer token. This keeps the security boundary in one place, lets each service scale independently, and means the same API could serve a future mobile or partner client unchanged.

4.2 Request lifecycle

1. A user signs in on the web app. **NextAuth** delegates credentials to the API's `POST /api/auth/login` , which returns a signed **JWT**. The token is carried in the NextAuth session and bridged to the browser.
2. **Server Components** render initial data by calling the API through `serverApi()` (which forwards the session's bearer token during SSR).
3. **Client "manager" components** mutate data through `apiFetch()` (bearer token from `localStorage`).
4. On the API, every request passes `requireAuth` (verify JWT) → `requirePermission` (RBAC check) → **Zod validation** → a **route handler**, which calls a **domain service**. Errors are mapped to HTTP by a single `onError()` handler.

4.3 Backend layering

The backend is deliberately layered so business logic stays testable and free of framework concerns:

Layer	Directory	Responsibility
Routes (52 modules)	src/routes/	HTTP surface. Authentication, permission gating, request validation, pagination/search/sort, and response shaping. Thin — they delegate to services.
Services (40 modules)	src/services/	Framework-independent domain logic: the double-entry engine, trip P&L, dispatch conflict checks, payroll statutory maths, FX revaluation, etc. Unit-tested in isolation.
Lib (cross-cutting)	src/lib/	prisma , auth (JWT + guards), rbac , scope (multi-tenancy), concurrency , activity (audit), validations (Zod), http (ok/created/onError), mail , password , rate-limit , format , errors .

A representative slice — creating a customer invoice — flows: routes/receivables.ts → validates with validations.ts → services/ap-ar.ts computes the posting → services/ledger.ts writes a **balanced** journal entry → lib/activity.ts records the audit event.

4.4 Frontend approach

- **Next.js App Router** with route groups: (auth) (login/reset), (admin) (the console, ~50 screens each with a loading.tsx skeleton), and (driver) (a locked-down portal).
- **Server Components** fetch initial data server-side via serverApi() ; **client components** ("managers") handle interactivity and mutations via apiFetch() .
- **middleware.ts** performs the auth check and the RBAC **route guard** before a page loads, and isolates drivers to /driver .
- The frontend holds **no Prisma/DB dependency**; reference data for dropdowns comes from the API's /api/lookups/* endpoints.

4.5 The accounting core

Every financial event becomes a **balanced double-entry journal** (services/ledger.ts):

- Debits must equal credits, or the entry is rejected.
- Posted entries are **immutable** — corrections are made with a contra-reversal, never by editing history (an audit requirement).
- Amounts use Decimal(18,2) (and finer scales for unit costs); an **exchange rate is stored on every posting** for multi-currency support.
- Operational actions post automatically: invoicing an order (Dr A/R · Cr Freight Revenue), trip expenses (Dr Expense · Cr A/P), POS sales, payroll, depreciation, inventory issues, and FX revaluation all route through the same engine.

This is why operations and finance cannot drift apart — there is one ledger and one set of rules.

5. Cross-cutting foundations

These patterns apply across every module and are the backbone of the system's integrity. They correspond to the PRD's non-negotiable Phase-1 requirements.

5.1 Multi-company (multi-tenancy)

A single database serves multiple legal entities. Every business record carries a **dataAreaId** string column, and isolation is **enforced at the query layer** — not left to callers — via src/lib/scope.ts :

- areaScope(user) — injects the tenant filter into reads.
- areaForWrite(user) — resolves the tenant a new record belongs to.
- assertSameArea(...) — guards cross-entity references.

A `Company` registry defines each entity (its `code` is the `dataAreaId`). Admins get a cross-entity view with a company switcher (the `X-Data-Area` header); non-admin cross-company reads return **404**. See also **M32/M34** below.

5.2 Dynamic role-based access control (RBAC)

Authorization is **data-driven and configurable at runtime**:

- Grants live in `rbac_roles` / `rbac_permissions` / `rbac_role_permissions` tables and hydrate an **in-memory map**, so `can()` stays synchronous and no route code changes when permissions are edited.
- **96 permissions** across the domains; **5 built-in system roles** (Admin, Dispatcher, Finance, Driver, Staff) that are protected from deletion but re-permissionable, plus fully **custom roles**.
- Enforced in **two layers**: the frontend `middleware` guards routes (using a mirrored copy in `frontend/lib/rbac.ts`), and the backend `requireAuth` + `requirePermission` guards **every API call** (the authoritative `backend/src/lib/rbac.ts`). The JWT carries the effective role key.

5.3 Audit trail

`src/lib/activity.ts` records every write to an `ActivityLog`. For key master data it captures a **field-level before → after diff** (`diffFields` / `logFieldChanges`) — only the columns that changed, normalized (dates → ISO, decimals → strings) — surfaced on the Activity Log screen.

5.4 Optimistic concurrency

Business models carry a `version` column; `src/lib/concurrency.ts` (`updateWithVersion()` / `requireVersion()`) rejects a stale update rather than silently overwriting a change made by someone else in the meantime.

5.5 Shared validation & enums

Zod schemas are the single validation contract for both API and forms. Because the two apps are separate, the schemas and the enum/label maps are **duplicated and kept in sync** (`backend/src/lib/validations.ts` ↔ `frontend/lib/validations.ts`; `frontend/lib/enums.ts`, `frontend/lib/labels.ts`).

5.6 Fiscal period close

A `FiscalPeriod` per company can be **closed**, after which `createJournalEntry` refuses any posting dated inside it — so **all** posting types (invoices, POS, payroll, revaluation...) respect the lock uniformly.

6. Module reference — functions in detail

Modules are grouped by domain. Each entry states the **function** (what it is for) and **implementation** (how it works, with the notable accounting/logic). For step-by-step verification of any module, see the [Verification Guide](#).

6.1 Accounting core

M8 · General Ledger

Function. The double-entry backbone every financial event posts to. **Implementation.** Balanced journals (debits = credits) with a Draft → Post → Reverse lifecycle; posted entries are immutable and corrected by contra-reversal. Dimensions and a per-posting FX rate are stored. `services/ledger.ts` is the single choke-point all automatic postings pass through.

M1 · Accounts Payable

Function. What the company owes suppliers, from bill to payment. **Implementation.** Vendor master (TIN/VRN, payment terms, withholding tax); enter → post → pay bills with ledger postings; integrates with Procurement's 3-way match (`services/ap-ar.ts`).

M2 · Accounts Receivable

Function. What customers owe, from invoice to receipt. **Implementation.** Customer registry (credit limit/days, tax-exempt, per-trip demurrage); raise → post → receipt invoices; invoicing a freight order posts Dr A/R · Cr Freight Revenue automatically.

M31 · Audit Workbench

Function. A dependable "who changed what, when" record. **Implementation.** `ActivityLog` on every write plus field-level before→after diffs on master data (§5.3), rendered on the Activity Log screen.

M34 · Organization Admin

Function. Run several legal entities on one installation and close their books per month. **Implementation.** `Company` registry + admin CRUD + company switcher; users have a home company; `dataAreaId` partitioning (§5.1); `FiscalPeriod` month-close enforced in the ledger (§5.6).

M36 · System Administration

Function. Manage users and design access. **Implementation.** User management (role, home company, active/inactive) and the dynamic RBAC engine (§5.2) with a resource-grouped permission matrix.

6.2 Core finance

M3 · Budgeting

Function. Spending limits per cost center, actively enforced. **Implementation.** CapEx/OpEx budget lines; control modes **STRICT_BLOCK** / **WARNING_ONLY** / **OVERRIDE**; consumed amounts wired into trip-expense and AP posting at the account level (`services/budget.ts`).

M4 · Cash & Bank

Function. Cash/bank/mobile-money accounts and driver disbursements. **Implementation.** Driver disbursements move PENDING → SUCCESS | FAILED | TIMEOUT and post to the ledger on settle (`services/cash-bank.ts`). *Live mobile-money/bank APIs are stubbed — see §9.*

M5 · Consolidations

Function. Roll subsidiaries up into a parent view. **Implementation.** Subsidiary→parent account mapping, rate-type rollup, and unmapped-account / missing-rate flags (`services/consolidation.ts`).

M6 · Cost Accounting

Function. Profitability at every grain — trip, corridor, contract. **Implementation.** `computeTripPnL` (per-trip P&L) + fuel efficiency per vehicle; per-corridor profitability (`getCorridorProfitability` , Corridor P&L screen); per-project P&L via M29; standard-cost variance via M13 (`services/cost-accounting.ts`).

M7 · Credit & Collections

Function. Chase overdue receivables and manage disputes. **Implementation.** Aging buckets (`agingBucket`), a dunning ladder (NONE → REMINDER → ... → LEGAL), dispute lifecycle, promise-to-pay, contact log, and bad-debt write-off (posts Dr 6200 · Cr 1100) (`services/collections.ts`).

M8 · Multi-currency revaluation

Function. Restate open foreign balances at period-end. **Implementation.** `revaluePeriod` re-states open foreign-currency AP/AR to the period-end rate and posts the net unrealized FX gain/loss (Dr/Cr 4900/6900) — preview then post from the Exchange Rates screen (`services/fx-revaluation.ts`).

M10 · Tax

Function. Prepare VAT/WHT returns per country. **Implementation.** Country tax-component tables for TZ/KE/UG/RW/ZM (`services/tax.ts` , `services/statutory.ts`). *Automated filing-period export is partial.*

6.3 Operations & logistics

M11 · Fleet Management

Function. Keep the fleet and drivers legal and efficient. **Implementation.** Vehicle registry + maintenance; insurance/inspection/COMESA/ Yellow Card expiries; driver documents; fuel-efficiency targets & monitoring; a unified compliance dashboard with a 14-day warning window (`services/compliance.ts` , `services/fuel.ts`).

M12 · Transportation Management

Function. Book and run cross-border freight. **Implementation.** Orders (corridor, weight/CBM, freight + demurrage, currency), Trips (route, mileage, expenses, automatic P&L), dispatch with conflict-checking, and freight-bill reconciliation (Unreconciled/Matched/Discrepancy) (`services/freight.ts` , `services/dispatch.ts`). *Live GPS feed is integration-gated.*

M33 · Master Planning

Function. Match forecast demand to fleet capacity. **Implementation.** Demand forecasts per period + corridor (loads & tonnage); a capacity plan comparing confirmed demand to available fleet (excluding maintenance, honoring truck overrides), surfacing per-corridor shortfall/surplus & utilization (`services/planning.ts`).

M30 · Common — GPS waypoints & units of measure

Function. Shared reference data: corridor map points and trading units. **Implementation.** A GPS **waypoint registry** (named checkpoints/borders/ weighbridges with lat/long, kind, country) and a **units-of-measure registry** (`services/uom.ts`) — a per-company catalog across weight/volume/distance/count, each with a factor to its dimension's base unit, plus a quantity converter that refuses cross-dimension conversions. Nine starter units are seeded.

6.4 Inventory & supply chain

M14 · Inventory

Function. Track parts, fuel, tyres and other stock at accurate cost. **Implementation.** Stock items with receipts/issues/adjustments, each posting to the ledger (Dr expense · Cr inventory); **moving-average** cost that updates on receipt; issues feed vehicle service orders (`services/inventory.ts`).

M13 · Cost Management

Function. Compare actual to target cost. **Implementation.** Moving-average valuation drives actual cost; a standard cost per item feeds the Cost Variance report (`getCostVarianceReport`) — per-unit and on-hand value variance, plus realized purchase-price variance from receipts (`services/cost-variance.ts`).

M15 · Procurement

Function. Purchase orders, receiving, and invoice matching. **Implementation.** PO lifecycle (DRAFT → APPROVED → PARTIAL → RECEIVED → CLOSED), goods receipts, and **3-way PO↔receipt↔invoice matching** with variance flags (`services/procurement.ts`).

M16 · Product Information

Function. Structured, searchable stock specifications. **Implementation.** A stock-item master plus a **product-attribute catalog** (`ProductAttribute` — TEXT/NUMBER/BOOLEAN/LIST, optional unit, scoped to a category or all) with a per-item value set from the inventory Specs editor (`services/product-attributes.ts`).

M18 · Warehouse

Function. Multi-location stock and transfers. **Implementation.** Per-location balances, a default warehouse, and inter-warehouse transfers (TRANSFER_OUT/IN).

6.5 Assets & workshop

M20 · Fixed Assets

Function. The register of owned assets and their depreciation. **Implementation.** Asset register (vehicle/equipment/...), straight-line depreciation runs posting Dr Depreciation Expense · Cr Accumulated Depreciation, and disposal with gain/loss (4910/6910).

M19 · Asset Management (custody & warranty)

Function. Track who holds an asset and its warranty status. **Implementation.** Assign an asset to a custodian (employee or department/site) with full history — **one open holder at a time** — and a warranty schedule (`warrantyExpiresAt`) surfaced on the register with CURRENT / EXPIRING_SOON / EXPIRED badges (30-day window) (`services/asset-custody.ts`).

M22 · Service Management

Function. The workshop — repair orders with parts and labor. **Implementation.** Service orders (OPEN → IN_PROGRESS → COMPLETED → POSTED); parts issued from Inventory; labor lines — external-garage labor accrues to the ledger, internal labor is a memo cost (`services/service-orders.ts`).

6.6 Human capital

M9 · Payroll

Function. Monthly payroll with correct statutory deductions. **Implementation.** Pay runs DRAFT → APPROVED → POSTED with country statutory computation (PAYE bands, NSSF, SHIF) and ledger posting; approved timesheet overtime (M26) is added to gross as taxable earnings (`services/payroll.ts` , `services/statutory.ts`).

M23 · Expense Management

Function. Trip cash-sheets and staff expense claims. **Implementation.** Claims DRAFT → SUBMITTED → APPROVED → POSTED with per-line receipt uploads, reconciled against driver advances, posted to the ledger (`services/expense.ts`).

M24 · Human Resources

Function. Contracts, leave, and employee documents. **Implementation.** Employment contracts (one ACTIVE per employee, salary sync on activation); leave management (working-day count, per-year balances, approve/cancel with balance refund); employee documents with 14-day expiry warnings (`services/hr.ts`).

M26 · Time & Attendance

Function. Clock-in/out rolled into timesheets that feed payroll. **Implementation.** Time entries rolled into monthly timesheets (regular vs overtime over 176h/mo); approved overtime feeds Payroll gross (`services/attendance.ts`). *Biometric-terminal ingestion is integration-gated.*

6.7 Commercial & retail

M27 · Sales & Marketing

Function. Win work and quote freight. **Implementation.** CRM leads (NEW → ... → WON/LOST) and freight quotations (`QUO-` sequence) with priced line items; an ACCEPTED quote converts **atomically** into a freight Order (M12), carrying the quoted total as freight revenue (`services/sales.ts`).

M28 · Retail / POS

Function. Over-the-counter sales of stock items. **Implementation.** Sales (`POS-` sequence); completing a sale relieves inventory at average cost and posts two balanced entries — Dr Cash · Cr Retail Revenue (4300) and Dr COGS (5300) · Cr Inventory (1300); oversell-guarded; void restores stock (`services/pos.ts`).

M29 · Project Management

Function. Group freight orders into contracts and track budget vs actual. **Implementation.** Projects (`PRJ-` sequence) grouping orders against a planned budget; a live budget-vs-actual P&L rolls up revenue (Σ freight + demurrage) and cost (Σ trip base costs + expenses) from linked orders; guarded lifecycle; attach/detach orders.

6.8 Analytics & KPI dashboard

Function. Turn the day's operations into the numbers managers watch. **Implementation.** The dashboard (`getKpiDashboard`) exposes KPIs across five categories — Operational & Delivery, Cost & Profitability, Fleet & Asset Utilization, Driver & Safety, and Customer Service. Pure formulas live in `services/dashboard-kpi.ts` (unit-tested); an impure aggregator feeds them. Three internal capture models feed the harder KPIs: `DockEvent` (→ truck turnaround), `DamageReport` (`DMG-` sequence → damage/claim rate), and `CustomerFeedback` (CSAT 1-5 / NPS 0-10), each with its own screen gated by `kpi:read` / `kpi:write` .

6.9 System & shared

M32 · Demo / Sandbox

Function. A safe, throwaway practice company. **Implementation.** `Company.isSandbox` flags an isolated demo partition (its own `dataAreaId`); an admin screen provisions it, shows headline counts, and **resets it to a clean demo baseline** — reset is **hard-guarded to refuse any non-sandbox company** — with a sandbox badge on the company switcher (`services/sandbox.ts`).

(M30, M31, M33, M34, M36 are documented in their functional domains above.)

6.10 Out of scope

M17 Vendor Collaboration, **M21 Production Control**, and **M25 Questionnaire** are intentionally not built — they do not apply to a logistics operation.

7. Security

Area	Implementation
Access control	Role→permission map enforced in two layers — frontend <code>middleware</code> (routes) + backend <code>requireAuth</code> + <code>requirePermission</code> (every API call).
Authentication	<code>bcrypt(12)</code> password hashing; stateless JWT bearer tokens issued by the API; NextAuth session carries and forwards the token on SSR and client requests.
Separation of duties	Accounting posting (Finance) is kept separate from operations editing (Dispatcher); user administration is Admin-only.
Input validation	Shared Zod schemas across all APIs and forms.
Accounting integrity	Entries must balance; posted entries are immutable (reverse via contra); <code>Decimal</code> precision.
Tenant isolation	<code>dataAreaId</code> enforced at the query layer; cross-company reads 404 for non-admins.
SQL injection	Prisma parameterized queries only; no raw SQL in application paths.
XSS / CSRF	React auto-escaping + strict <code>Content-Type</code> ; NextAuth CSRF token + SameSite cookies.
Rate limiting	<code>lib/rate-limit.ts</code> on login, password reset, and create operations.
Uploads	MIME/size validation, randomized file names.
Audit	All writes recorded in <code>ActivityLog</code> , with field-level diffs on master data.

8. Quality & testing approach

- **Unit tests (Vitest).** 35 test files cover the domain **services** — the parts where correctness matters most: double-entry balancing, trip P&L, dispatch conflict detection, payroll statutory maths, UoM conversion, warranty/aging classifiers, validation, and error mapping. Services are framework-independent precisely so they can be tested directly.
- **Type safety.** Both apps are fully type-checked (`tsc --noEmit`); the frontend additionally runs a production build check.
- **Integration & end-to-end.** Integration tests run against a real database (`migrate deploy` + seed). New modules are additionally validated with an ephemeral PostgreSQL end-to-end script and a live UI pass before sign-off.
- **Seed data.** `prisma/seed.ts` provisions the chart of accounts, users per role, demo orders/trips, waypoints, and starter units — so a fresh install is immediately explorable.

9. External integrations

The data models are already in place to **receive** these connections (e.g. `MoneyTransfer.status` , `externalRef`); no live API clients are implemented yet. They are gated on client-provided sandbox credentials and represent the natural next milestone:

- **E-invoicing** — KRA TIMS (Kenya), TRA VFD (Tanzania)
- **Mobile money** — M-Pesa, Airtel Money
- **Bank feeds** — CRDB / KCB / Stanbic statement import
- **GPS telematics** — live vehicle positions & idle time (the waypoint registry works)
- **Biometric terminals** — automated clock-in (manual entry works)

10. Delivery status

Phase	Scope	Status
1 — Foundation	RBAC, GL, AP/AR, multi-company, audit	✓ Done
2 — Core Finance	Budgeting, cash & bank, tax, consolidation, multi-currency	✓ Done
3 — Operations	Fleet, transport, GPS, common	✓ Done
4 — Inventory & Payroll	Inventory, warehouse, procurement, payroll, expense	✓ Done
5 — Assets & HR	Fixed assets, service, HR, time & attendance	✓ Done
6 — Commercial & Dashboard	Sales, POS, projects, cost accounting, planning, dashboard	✓ Done

33 of 36 modules delivered and protected by 266 automated tests. The 3 remaining are out of logistics scope. Next: UAT sign-off, then enabling the external integrations in §9. The authoritative, continuously-updated map is [PRD-STATUS.md](#).

11. Repository structure

```
fleetERP/
├─ backend/                # standalone Hono API – owns the database
│  ├─ prisma/
│  │  ├─ schema.prisma    # 78 models, 69 enums
│  │  ├─ migrations/      # 35 migrations
│  │  └─ seed.ts          # users, chart of accounts, demo data, starter units
│  └─ src/
│     ├─ main.ts          # Hono bootstrap; mounts 53 route groups under /api
│     ├─ routes/          # 52 HTTP modules (thin; auth + validation + shaping)
│     ├─ services/        # 40 domain modules (the business logic; unit-tested)
│     └─ lib/             # prisma, auth, rbac, scope, concurrency, activity,
│                          # validations, http, mail, password, rate-limit, ...
├─ tests/ unit/ + integration/
├─ frontend/              # standalone Next.js app – no DB dependency
│  ├─ middleware.ts       # auth + RBAC route guard
│  ├─ auth.ts            # NextAuth; login delegates to the backend
│  ├─ app/ (auth)/ · (admin)/ (~50 screens) · (driver)/
│  ├─ components/ ui/ (shadcn) · layout/ · data/
│  └─ lib/ server-api · fetcher · rbac · enums · labels · validations · api-types
└─ docs/                  # this file + PRD-STATUS, VERIFICATION-GUIDE,
                        # DATABASE, API, DEPLOY, MIGRATION
```

Last updated: 2026-08-13. The code is the source of truth; this document is maintained by hand — re-verify counts and specifics before external sign-off.